Diseño PCAM previo al código: simulación N-Body 2D

David Domínguez (23712), Jonathan Diaz (23873) y Luis Padilla (23663)

Repositorio: https://github.com/Jonialen/labPCAM

1. Problema, entrada e invariantes

Cada cuerpo tiene posición (x,y), velocidad (vx,vy) y masa. Se reciben N, pasos, número de hilos, política de asignación y chunk. La semilla 42 produce el mismo estado inicial para comparar versiones. Se usa suavizado eps=0.01, G=1 y dt=0.001.

- Dentro de un paso, actual es inmutable: todas las fuerzas usan la misma fotografía del sistema.
- siguiente[i] se escribe una sola vez después de sumar todas las contribuciones a i.
- La velocidad se actualiza antes que la posición. La masa se conserva.
- El paso t+1 no empieza hasta que todos los cuerpos del paso t estén listos.

2. P — Partition: identificar el paralelismo posible

La partición es por datos. Una contribución de j sobre i es la unidad fina de cálculo en la versión directa; la versión simétrica toma cada pareja no ordenada (i,j) con i<j una sola vez. Las contribuciones de un mismo paso pueden calcularse en paralelo si se combinan sin carreras. La actualización temporal introduce una dependencia entre pasos. Separar cálculo de fuerzas e integración en fases no permite ejecutar ambas simultáneamente para un mismo paso.

3. C — Communication: datos, combinación y sincronización

Todos los hilos leen actual. En la versión directa, cada hilo suma las contribuciones de un cuerpo en variables locales y escribe solo su siguiente[i]: no hay escrituras compartidas sobre la fuerza. En la versión simétrica, una pareja aporta a i y a j. Si varios hilos escribieran los mismos acumuladores aparecería una carrera; cada hilo acumula en un plano privado y después se hace una reducción por cuerpo. Una barrera termina el cálculo de parejas antes de la reducción, y otra termina la integración antes de avanzar al siguiente paso.

4. A — Agglomeration: elegir tareas prácticas

No se crea una tarea OpenMP por pareja. En la directa se agrupan N-1 contribuciones en una iteración del cuerpo i. En la simétrica se agrupan N-i-1 parejas en una iteración de i. Esto reduce el costo de administrar tareas, aunque la versión simétrica deja iteraciones de tamaño decreciente. Un chunk agrupa además varias iteraciones exteriores en una asignación. Con static, cambiar el chunk también cambia el reparto entre hilos: la prueba de chunks evalúa A y M conjuntamente.

5. M — Mapping: asignar tareas a hilos

El bucle exterior se asigna a los hilos con static, dynamic o guided. static fija el reparto; dynamic entrega el siguiente chunk cuando un hilo queda libre; guided reduce gradualmente el tamaño de los bloques. En la directa, cada i examina aproximadamente N-1 cuerpos; en la simétrica, el trabajo por i cae de N-1 a cero. Se compararán 2, 4 y 8 hilos y varios chunks. La política elegida debe valorar el tiempo ocioso y el overhead de scheduling.

6. Pseudocódigo: referencia secuencial

```
actual <- inicializar(N, seed=42)
siguiente <- reservar(N)
repetir pasos veces:
    para i = 0 .. N-1:
        ax, ay <- 0, 0
        para j = 0 .. N-1, j != i:
            dx, dy <- actual[j].pos - actual[i].pos
            r2 <- dx*dx + dy*dy + eps*eps
            f <- G * actual[j].masa / (r2 * sqrt(r2))
            ax, ay <- ax + f*dx, ay + f*dy
        siguiente[i].v <- actual[i].v + (ax,ay)*dt
        siguiente[i].pos <- actual[i].pos + siguiente[i].v*dt
        siguiente[i].masa <- actual[i].masa
    intercambiar(actual, siguiente)
```

Esta referencia define el resultado esperado. La comparación paralela utiliza el mismo estado inicial y el mismo orden de actualización velocidad-posición.

7. Pseudocódigo: versión paralela directa

```
actual <- inicializar(N, seed=42); siguiente <- reservar(N)
repetir pasos veces:                        // [P] pasos dependientes
    paralelo_para i = 0 .. N-1:              // [A] tarea = cuerpo i
      schedule(politica, chunk)              // [M] asignación a hilos
        ax, ay <- 0, 0                       // [C] acumuladores locales
        para j = 0 .. N-1, j != i:           // [P] aporte fino (i,j)
            dx, dy <- actual[j].pos - actual[i].pos
            r2 <- dx*dx + dy*dy + eps*eps
            f <- G * actual[j].masa / (r2 * sqrt(r2))
            ax, ay <- ax + f*dx, ay + f*dy
        siguiente[i].v <- actual[i].v + (ax,ay)*dt
        siguiente[i].pos <- actual[i].pos + siguiente[i].v*dt
        siguiente[i].masa <- actual[i].masa  // [C] escritura exclusiva
    barrera                                  // [C] estado completo
    intercambiar(actual, siguiente)
```

La barrera de parallel for protege el intercambio. El chunk agrupa varias tareas i; su tamaño se prueba experimentalmente.

8. Pseudocódigo: versión simétrica por parejas

```
actual <- inicializar(N, seed=42); siguiente <- reservar(N)
repetir pasos veces:                              // [P] pasos dependientes
    parcial[hilos][N] <- (0,0)                     // [C] planos privados
    region_paralela con T hilos:
        t <- id_del_hilo
        paralelo_para i = 0 .. N-1:               // [A] parejas agrupadas por i
          schedule(politica, chunk)                // [M] reparto de i
            para j = i+1 .. N-1:                   // [P] pareja única (i,j)
                dx, dy <- actual[j].pos - actual[i].pos
                r2 <- dx*dx + dy*dy + eps*eps
                f <- G / (r2 * sqrt(r2))
                parcial[t][i] += f * masa[j] * (dx,dy)
                parcial[t][j] -= f * masa[i] * (dx,dy)
        barrera                                    // [C] pares completos
        paralelo_para i = 0 .. N-1:
            (ax,ay) <- suma_t(parcial[t][i])       // [C] reducción
            siguiente[i] <- integrar(actual[i], ax, ay, dt)
        barrera                                    // [C] estado completo
    intercambiar(actual, siguiente)
```

parcial[t][i] representa las dos componentes de aceleración; la implementación las almacena en dos planos. integrar aplica exactamente el orden de la referencia: velocidad nueva, posición nueva y masa conservada. Los planos privados evitan atomic por pareja. Se requieren O(T*N) acumuladores adicionales; la reducción también cuesta O(T*N). La versión simétrica calcula N(N-1)/2 parejas por paso, pero podría tardar más que la directa si esos costos o el desbalance dominan.

9. Plan de verificación y medición

- Corrección: misma semilla y parámetros en todas las versiones; caso de un cuerpo sin fuerza propia; caso de dos cuerpos con fuerzas opuestas; comparación de todos los campos contra la referencia secuencial con tolerancia numérica.
- Concurrencia: ejecutar varios hilos y políticas, comprobar que cada salida pertenece a un solo hilo y que la reducción simétrica ocurre después de la barrera.
- Rendimiento: N=5000, 10 pasos, seed=42; al menos tres corridas por configuración. Medir solo la simulación con omp_get_wtime(). Registrar tiempos crudos, promedio y desviación.
- Comparación: speedup S(p)=T1/Tp y efficiency E(p)=S(p)/p. Comparar 2, 4 y 8 hilos, static, dynamic, guided, al menos tres chunks y la directa frente a la simétrica con configuración equivalente.

Las mediciones servirán para decidir qué asignación funciona mejor en este equipo; el diseño no presupone que menos parejas calculadas implique menor tiempo total.